

Kubernetes Notes

Your Name

Date:

Contents

Chapter 1

Notes info and Introduction

Purpose of notes There purpose of these notes are to gain an understanding of Docker and have reference material in case I decide to sit the DCA exam.

Chapter 2

Core Concepts

2.1 Kubernetes Architecture

Nodes A node is a machine, physical or virtual upon which Kubernetes is installed. This worker machine is where containers will be launched by Kubernetes.was known as minions in the past... These terms may be used interchangeably.

Consider a node failing.....application will go down.....hence we need multiple nodes.

Cluster A cluster is a group of nodes grouped together. This way when one node fails, your application is still accessible from the other nodes. Having multiple nodes helps in sharing the load as well.

Master node Things to consider - What is responsible for managing the cluster - Where is the information about members of the cluster stored? - How are nodes monitored? - When a node fails, how do you move the workload of the failed node to another worker node?

This is the purpose of the master. The master is another node with Kubernetes installed in it and is configured as a master. The master watches over the nodes in the cluster and is responsible for the actual orchestration of containers on the worker nodes.

2.1.1 What is installed on a system when installing Kubernetes?

- API Server
- etcd
- Kubelet
- Container Runtime
- Controller
- Scheduler

API Server

An API server which acts as the frontend for Kubernetes. The user management devices, command line interfaces all talk to the API server to interact with the Kubernetes cluster.

etcd keystore

Etcd is a distributed, reliable key value store used by Kubernetes to store all data used to manage the cluster. The way to think of it is, when you have a multiple nodes and masters in your cluster, etcd stores all that information on all the nodes in the cluster in a distributed manner Etcd is responsible for implementing locks within the cluster to ensure that there are no conflicts between the masters,

The Scheduler

is responsible for distributing work or containers across multiple nodes. It looks for newly created containers and assigns them to nodes.

Controllers

The "brain" behind orchestration. They are responsible for noticing and responding when nodes, containers or endpoints go down. The controllers decide whether to bring up new containers.

Container Runtime

They are responsible for noticing and responding when nodes, containers or endpoints runtime is the underlying software that is used to run containers. An example is Docker, but alternatives exist.

Kubelet

This is the agent that runs on each node in the cluster. The agent is responsible for making sure that the containers are running on the nodes as expected.

There are two types of server, masters and a worker along with a setup of components which make up Kubernetes.

How does one server become a master and the other the slave.

The worker node, or minion as it is also known, is where the containers are hosted.

The master has the Kube API server and that is what makes it a master. Similarly the worker nodes have the Kubelet agent that is responsible for interacting with master to provide both health information of the worker node and carry out actions requested by the master on the worker nodes.

All the information gathered is stored in a key value store on the master. The key value store is based on the popular etcd framework.

The master also has the control manager and the scheduler.

Kube command line or kubectl or kube control

The kube control tool, is used to deploy and manage applications on Kubernetes cluster to get cluster information, to get the status of other nodes in the cluster. And to manage other things. The kubectl cluster info command is used to view information about the cluster and the kubectl get nodes command is used to list all the nodes as a part of the cluster.

2.2 Docker-vs-ContainerD

Initially kubernetes was the container runtime for docker only. Support for other container runtimes was added. This was done via an interface called Container Runtime Interface or CRI.

CRI allows any vendor to work as a container runtime for Kubernetes, as long as they adhere to the OCI standards. OCI stands for the open container initiative and it consists of an image spec and runtime spec.

Image spec means the specifications on how an image should be built. An image spec defined the specifications on how an image should be built and the runtime spec defined the standards on how nay container runtime should be developed.

Keeping the OCI standards in mind. Anyone can build a container runtime that can be used by anybody to work with Kubernetes Note: Docker wasn't built for CRI. To maintain support between kubernetes and Docker, Kubernetes introduced dockershim. Docker runs without CRI.

Docker isn't just a container runtime along. Docker consists of multiple tools that are put together. - Docker CLI, the Docker API and the build tools that help in building images. There was support for volumes or security. There is a container runtime called run C and that managed a container called container D Container D can be used as a runtime on its own separated from Docker.

...dockershim and docker support was removed from kubernetes but docker follows the OCI standards now.

Container D is a part of Docker but is a separate project on its own. You can run container D without having to install Docker itself. If you don't need dockers other features you could ideally just install container D alone.

CLI - ctr

CLI - nerdctl

nerdctl provides a docker like CLI for containerd nerdctl supports Docker compose nerdctl supported newest features in containerd - Encrypted container images - Lazy pulling - P2P image distribution - image signing and verifying - Namespaces in Kubernetes

CLI-crictl

crictl provides a CLI for CRI-compatible container runtimes installed separately used to inspect and debug container runtimes not to create container ideally works across different runtimes

2.3 Pods

Containers are encapsulated into a Kubernetes object know as pods. A pod is a single instance of an application. A pod is the smallest object you can create in Kubernetes.

Can increase the number of pods to scale the application. Can add a new node and add pods to that if the current nodes do not have capacity.

Pods usually have a 1 to 1 relationship with You do not add containers to a single pod to scale your applicaiton.

2.3.1 Muti-Container Pods

You can have multiple containers in the same pod but they will not be the same kind. Example: helper container. The two containers in the same pod can communicate with each other over localhost. They can easily share the same storage space and the same network space.

2.4 YAML Basics

2.5 Pods with YAML

Kubernetes uses YAML files as inputs for the creation of objects such as pods, replicas and deployments, services.

2.5.1 YAML in Kubernetes

- `apiVersion`: This is the version of the Kubernetes API we are using to create the object. Must use the correct API version for the objects we are trying to create. See other options.
- `kind`: This refers to the type object we are trying to create. Options are Pod, Service, ReplicaSet and Deployment.
- `metadata`: This is data about the object like its name, labels, etc.
- `spec`:

Listing 2.1: Pod commands

```
kubectl create -f pod-definition.yml
kubectl get pods
kubectl describe pod myapp-pod
```

Listing 2.2: Example YAML Configuration

```
database:
  host: localhost
  port: 5432
  username: admin
  password: secret

features:
  logging: true
  cache: false

servers:
  - app01.example.com
  - app02.example.com
```

2.5.2 ReplicatSets

Replication Controller

The replication controller helps us run multiple instances of a single pod in the Kubernetes cluster allowing for High Availability. This can be done with a single pod. The replication controller can help by automatically bring up a new pod when the existing one fails. The replication controller ensures that the specified number of pods are running at all times.

Another reason we need replication controllers is to create multiple pods to share the load across them.

The replication controller allows us to create multiple pods to share the load across them. The replication controller spans across multiple nodes in the cluster. It allows for the application to be scaled when the demand increases.

Note: there are two similar terms. Replications Controller and Replica Set. Both have the same purpose, but they are not the same. Replication controller is the older technology that is being replaced by the replicat set.

Replicat set is the new recommended way to set up replication. There are minor differences in the way each works.

How to create a replication controller

Start by creating a rc-definition.yml YAML file. This will have the standard info as any Kubernetes file. API, version, kind, metadata and spec. The API version is specific to what we are creating. The kind is set to replication controller. Add a name under metadata. Add labels and assign values. Add the spec to the YAML. The spec defines what is inside the object we are creating. Create a template section under the pod template to be used by the replication controller to create replicas.

How is the pod template defined? Use pod definition file metadata.

Listing 2.3: Replication controller

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      container:
        - name: nginx-container
          image: nginx
  replicas: 3
```

Listing 2.4: Pod commands

```
kubectl create -f rc-definition.yml
```

```
kubectl get replicationcontroller
kubectl get pods
```

Listing 2.5: Replication Set

```
apiVersion: apps/v1
kind: ReplicationSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metedata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      container:
        - name: nginx-container
          image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

The selector section helps the replica set identify what pods fall under it. Replica sets can also manage pods that were no created as a part of the replica set creation. Example, pods created before the creation of the replica sets that match the labels specified in the selector. The replicaset will also take those pods into consideration when creating the replicas.

The selector is one of the major differences between replication controller and replica set. The selector is not a required field in case of a replication controller, but it is still available.

Match label selector simply matches the labels specified under it to the labels on the pods.

Labels and selectors

Uses of replica sets. Use them to monitor existing pods if they have already been created.

The role of the the replica set is to monitor the piods and if any of them were to fail, deploy new ones. The replica set is a process that monitors the pods.

Labelling the pods is handy since we can now use these as a filter for replica set under the selector section, we use the match labels filter and provide the same filter we used while creating the pods.

Scaling with a replica set

One way is to update the number of replicas in the definition file to six replicas the run kubectl's apply command to sspecifiy the same file using the f parameter and the will update the replica set to have six replicas.

Another way is to run the kube control scal command. Use the replicas parameter to provide the number of replicas and specify the same file as input. - You can either input the definition file

or provide the replica set name in the type name format. However, remember that using the file name as input will not result in the number of replicas being updated automatically in the file. — In other words the number of replicas in the replica set definition file will still be three, even though via this command it was scaled to six.

There are also options available for automatically scaling the replica set based on load, but that is an advanced topic.....

Listing 2.6: Pod commands

```
kubectl create -f replicaset-definition.yml
kubectl get replicaset
kubectl delete replicaset myapp-replicaset
kubectl replace -f replicaset-definition.yml
kubectl scale --replicas=6 -f replicaset-definition.yml
```

Deployment

Each container is encapsulated inside a pod. Multiple pods are deployed using replication controllers or replica sets.

Higher in the hierarchy of Kubernetes, above replica sets are deployments

Listing 2.7: Replication Set

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
    type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      container:
        - name: nginx-container
          image: nginx
  replicas: 3
  selector:
    matchLabels:
      type: front-end
```

Listing 2.8: Pod commands

```
kubectl create -f deployment-definition.yml
kubectl get deployments
kubectl get replicaset
kubectl get pods
kubectl get all
```

Namespaces

Kubectl Explain Command

Pods, deployments and clusters are deployed within a namespace.

Kubernetes creates a default namespace when the cluster is setup and these are created under a namespace called kubesystem. The kube-public namespace is where resource that should be made available to all users are created.

When a kubernetes cluster grows to production/enterprise level you will want to consider using namespaces.

You can create your own namespaces as well. Example, if you want to use the same cluster for both dev and production environments but at the same time isolate the resources between them. You can create a different namespace for each of them.

That way while working in the dev environment, you don't accidentally modify resources in production.

Each of these namespaces can have its own set of policies that define who can do what.

You can also assign a quota of resources to each of these namespaces. That way, each namespace is guaranteed a certain amount and does not use more than its allowed limit.

Resources within a default namespace can refer to each other by their names.

You can make sure that the pod gets created into the development environment all the time by moving the namespace into the pod definition file.

Listing 2.9: Replication Set

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: myapp-pod
  namespace: dev
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Listing 2.10: Pod commands

```
kubectl get pods
kubectl get pods --namespace=kube-system

kubectl create -f pod-definition.yml
kubectl create -f pod-definition.yml --namespace=dev
```

Create Namespace

Listing 2.11: Replication Set

```
apiVersion: apps/v1
kind: Namespace
```

```
metadata:
  name: dev
```

Listing 2.12: Pod commands

```
kubectl create -f namespace-dev.yml
kubectl create namespace dev
kubectl get pods --namespace=dev
#Used to change context for namespace command
kubectl config set-context $(kubectl config current-context) --namespace=dev

kubectl get pods
kubectl get pods --namespace=default
kubectl get pods --namespace=prod

# view pods in all namespaces
kubectl get pods --all-namespaces
```

Resource Quota

Listing 2.13: Replication Set

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev

spec:
  hard:
    pods: "10"
    request.cpu: "4"
    requests.memory: 5Gi
    limits.cpu: "10"
    limits.memory: 10Gi
```

Listing 2.14: Pod commands

```
kubectl create -f compute-quota.yaml
```

Services

Kubernetes services enable communication between various components within and outside of the application. These services help connect application together with other applications or users.

Example to consider. Application has a group of pods running various sections such as group for running back end processes, a group for frontend services and a group connecting to an external data source.

Services enabled the front end application to be made available to end users. It helps communication between backend and frontend pods and helps in establishing connectivity to an external data source.

Services enable loose coupling between microservices in our application.

A kubernetes service is an object just like pods, replica sets or deployments that we worked with before. One of its use cases is to listen to a port on the node and forward requests on that port to a port on the pod running the web application.

This type of service is known as a node port services, because the service listens to a port on the node and forwards the request to the pods.

Service Types

There are different types of services.

The first type is the node port where the service makes an internal port accessible on a port on the node.

The second is cluster IP and in this case the service creates a virtual IP inside the cluster to enable communication between different services, such as a set of front end servers and a set of backend servers.

The third type is a load balancer, where it provisions a load balancer for our applicaiton in supported cloud providers.

Node Port

There are three ports involved. The port on the pod where the actual web server is running. This is referred to as the target port because that is where the service forwards the request. The second port is the port on the service. Referred to as the port. This service is in fact like a virtual server inside the node, inside the cluster. It has its own IP address and that IP address is called the cluster IP of the service. Finally, we have the port on the node itself which we can use to access the web server externally. -¿ this is known as the node port. Node ports can only be in a valid range which is by default from 30,000 to 32,767.

How to create a service

Use a definition file to create a service. The high level structure of the file remains the same as before.

Listing 2.15: Replication Set

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service

spec:
  type: NodePort
  ports:
    - targetPort: 80 #Note that this is an array - you can have multiple p
      port: 80 #Out of all of these this is the mandatory port
      nodePort: 30008
  selector: # Use to identify the pod
    app: myapp
    type: front-end
```

If you don't provide a target port, it is assumed to be the same as port. Also, if you don't provide a node port, a free port in the valid range between 30,000 and 32,767 is automatically allocated.

Listing 2.16: Pod commands

```
kubectl create -f service-definition.yml
kubectl get services
# We can now this port to access the web service using Curl or a web browser
curl http://192.168.1.2:30008
```

Requests are randomly routed to pods.... The service acts as a built-in load balancer to distribute load across different pods

In this case we have web applications in pods on separate nodes in a cluster. Kubernetes creates a service which spans all the nodes in the cluster.

They can be accessed using curl with the correct IP address and port.

To summarise in either case the service is created exactly the same without having to do any additional steps during the service creation.

When pods are removed or added, the service is automatically updated, making it highly flexible and adaptive. Once created, you won't typically have to make any additional configuration changes.

Services - Cluster IP

A full stack web application typically has different kinds of pods hosting different parts of an application. You may have a number of pods running a frontend web server, another set of pods running a backend server, a set of pods running a key value store like redis etc.

They will need to communicate with each other.

What is the right way for them to communicate?

The pods all have an IP address assigned to them but these IPs as we know are not static. These pods can go down any time and new pods are created all the time.

You cannot rely on these IP addresses for internal communication between the application.

A Kubernetes service can help us group the pods together and provide a single interface to access the pods in the group.

Example: A service created for the backend pods will help group all the backend pods together and provide a single interface for other pods to access this service. The requests are forwarded to one of the pods under the service randomly.

Similarly create additional service for readings and allow the backend pods to access the reading systems through the service.

This enables us to easily and effectively deploy a microservices based application on a Kubernetes cluster.

Each layer can now scale or move as required without impacting communication between various services.

Each service gets an IP and name assigned to it inside the cluster and that is the name that should be used by other paths to access the service.

This type of service is known as the cluster IP

Listing 2.17: Cluster IP service definition

```
apiVersion: v1
kind: Service
metadata:
  name: back-end
spec:
  type: ClusterIP # This is default
```

```
ports:
  - targetPort: 80 # Where the backend is exposed
    port: 80 # Where the service is exposed
selector: # To Link a service to a set of ports
  app: myapp
  type: back-end
```

Listing 2.18: Commands

```
kubectl create -f service-definition.yml
kubectl get services
```

Kubectl Command

SEE DOCS kubectl cheat sheet and kubectl overview

Listing 2.19: Commands

```
kubectl api-resources
kubectl explain pods #top level
kubectl explain pods --recursive # Easier to create YAML files out of this
```

Chapter 3

Configuration

3.1 Define build and modify container images

3.2 Commands and Arguments in Docker

3.3 Commands and Arguments in Kubernetes

3.4 Environment Variable

3.5 ConfigMaps

3.6 Secrets

3.7 Pre-requisite - Security in Docker

3.8 Security Contexts

Chapter 4

Multi-Container Pods

Chapter 5

Observability

Chapter 6

Pod Design

Chapter 7

Services and Networking

Chapter 8

State Persistence

Chapter 9

Security

Chapter 10

Helm Fundamentals

Chapter 11

Kustomize